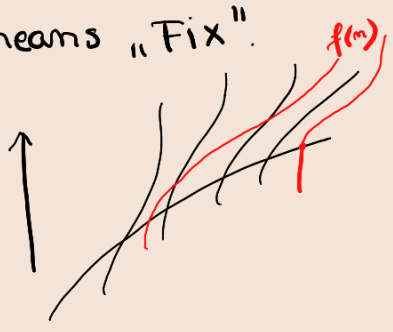
1) Def: A problem = a finite set of input and output variables.

$$P = \{ v_{i=\overline{0,n-1}} \in D_{v_i} \}$$

2) Brute force : the algorithm that solves any problem.

```
for α0 in Dv0 : |Dv0|
    for α1 in Dv1 : |Dv1|
        ⋮
        for αk-1 in Dvn-1 : |Dvn-1|
            candidate = (α0, α1, ..., αn-1)
            if is_solution(candidate)
```

3) Θ means „Fix".

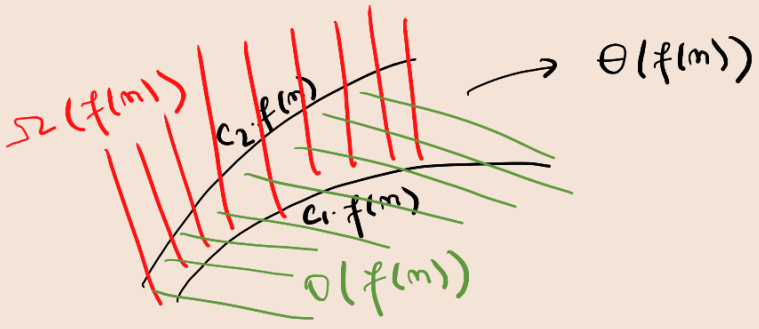


Complexitate bună: când f crește încet.

$\Omega(f(n))$ = totalitatea funcțiilor pe care le-am desenat
- → ele cresc mai rapid la ∞
- → ele cresc mai rapid sau la fel cu $f(n)$

$O(f(n))$ = totalitatea funcțiilor care cresc mai încet la ∞
- → ele cresc mai încet sau la fel cu $f(n)$
- → „se poate mai bine"

4) Înmulțim cu o constantă: $c_2 > c_1 > 0$ (scalari / constante)



$$\Omega(f(n)) \cap O(f(n)) = \Theta(f(n))$$

5) $N_0\ N_1\ N_2\ N_3 \Rightarrow |D_{N_0}| \times |D_{N_1}| \times |D_{N_2}| \times |D_{N_3}| = 10^4$ (each $|D_{N_i}| = 10$)

$\in \{0, \dots, 9\}$

6) Eficientizare:

```
candidate = ∅
for α_0 in D_{N_0}:
    candidate.append(α_0)
    for α_1 in D_{N_1}
        candidate.append(α_1)
        :
        for α_{k-1} in D_{N_{m-1}}
            candidate.append(α_{m-1})
            if is_solution(candidate)
                ...
            pop()
        pop()
    pop()
```

if is_consistent(candidate) BACKTRACKING ← 50% T, 50% F ⇒ DIVIDE ET IMPERA

mai puțini pași ⇓ $O(f(n))$

↳ isn't that supposed to be the worst case?

↳ Tabel: bag în listă cu .append (lipește)

→ Append: pune în coada unei liste o valoare

→ No more Θ, but O.

{ Backtracking is as efficient as you make it.
Not backtracking, but brute force is inefficient.

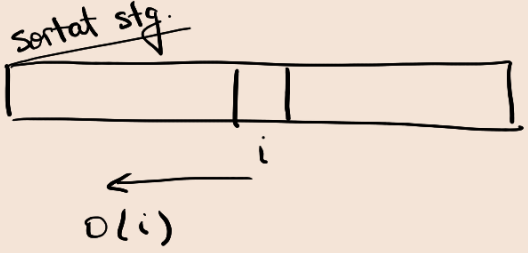
Θ —backtracking→ O

(initial complexity)

7) Selection sort works on brute force: $\Theta(n^2) = \frac{n(n+1)}{2}$

Bubble sort] → alg. de sortare cu $O(n^2)$ și care folosesc btrak.

Insertion sort
↳ tot ce e în stg. e sortat

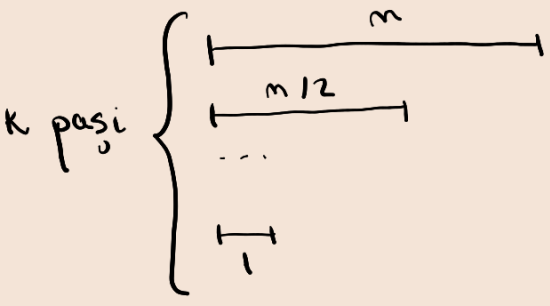


$$\sum_{i=0}^{n-1} O(i) = O\left(\sum_{i=0}^{n-1} i\right) = O(n^2)$$

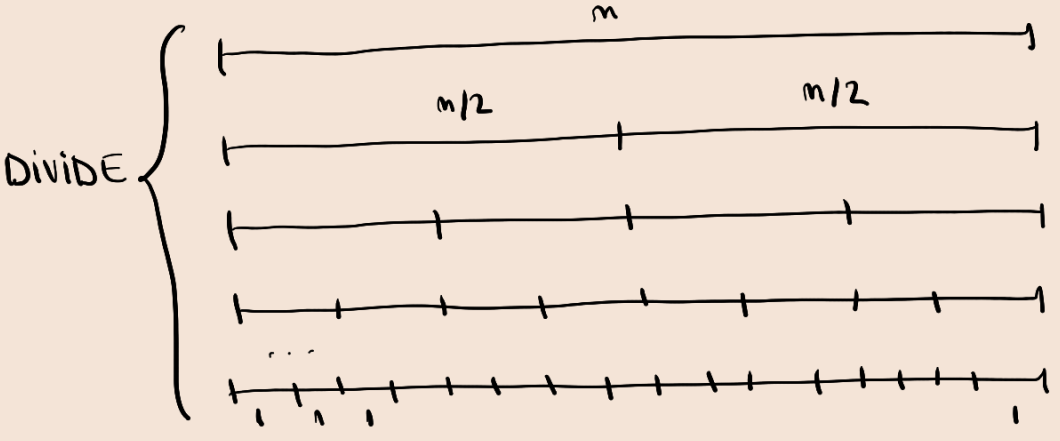
$\Theta(n \cdot \log_2 n)$ - Merge Sort

$O(n \cdot \log_2 n)$ - Heap Sort
- Quick Sort

8) DIVIDE ET IMPERA



$\Rightarrow \frac{n}{2^k} = 1 \Rightarrow n = 2^k \Rightarrow k = \log_2 n.$



→ liste multe cu un singur element =>
=> sortate deja.